Siddharth Bhat

siddharth.bhat@cl.cam.ac.uk · +44 7442 785227 · pixel-druid.com · Churchill College, Cambridge, UK
github.com/bollu · linkedin.com/in/siddharth-bhat · Google Scholar · UK Global Talent visa (Exceptional Promise)

Summary

ML Performance Engineer at **XTX Markets**. Compiler engineer and researcher (PhD, Cambridge) with a decade of performance-critical C++ inside production compilers and low-level runtimes. #3 contributor to **Polly**, LLVM's polyhedral loop optimizer (121 commits, 6000 LoC of C++ to LLVM, Fortran support and unified-memory CUDA codegen); primary author of **Lean's LLVM backend**, lowering an IR through instruction selection and code generation down to native machine code; #2 contributor to **Asterius**, a Haskell→WebAssembly compiler whose runtime work was merged into GHC. Five talks at LLVM developer meetings on IR design, dominance, side-effect modelling and optimization verification. Comfortable reasoning about the machine underneath the language — cache behaviour, memory models, SIMD, assembly — and optimizing hot code to that level. Fluent in C++, Lean, Haskell, Rust, Rocq.

Expertise

- **Loop optimization & HPC (C++, LLVM, CUDA):** #3 contributor to Polly — GPU code generation for Fortran workloads, unified-memory support in the CUDA backend, polyhedral scheduling; tiling patterns and stencil optimization in PolyMage, PLUTO and ISL (SC 2017); LLVM GSoC mentor (2016, 2018).
- **Backends & runtimes:** primary author of Lean's LLVM backend; reimplemented the GHC runtime on top of WebAssembly for Asterius (merged upstream into GHC); lock-free atomic synchronization for real-time audio in the PPSSPP C++ emulator; rewrote a raytracer in Haskell to match hand-tuned C++ (Haskell Exchange '20).
- **IR design & semantics (MLIR):** led `lean-mlir`, formal semantics for MLIR with machine-checked peephole rewriting (ITP 2024); designed SSA-based IRs for functional (CGO 2022) and quantum (CC 2021) compilation; verified translation validation for rewrites at arbitrary bitwidth.

Selected Publications

1. Verifying Peephole Rewriting in SSA Compiler IRs. *Siddharth Bhat*, Alex Keizer, Chris Hughes, Andres Goens, Tobias Grosser. *ITP 2024*.
2. Lambda the Ultimate SSA: optimizing functional programs in SSA. *Siddharth Bhat*, Tobias Grosser. *CGO 2022*.
3. Guided Equality Saturation. Thomas Koehler, Andres Goens, *Siddharth Bhat*, Tobias Grosser, Phil Trinder, Michel Steuwer. *POPL 2024*.
4. QSSA: an SSA-based IR for Quantum Computing. Anurudh Peduri, *Siddharth Bhat*, Tobias Grosser. *CC 2021*.
5. Optimizing Geometric Multigrid Computation using a DSL Approach. Vinay Vasista, Kumudha KN, *Siddharth Bhat*, Uday Bondhugula. *Supercomputing (SC) 2017*.
6. Certified Decision Procedures for Width-Independent Bitvector Predicates. *Siddharth Bhat*, Léo Stefanesco, Chris Hughes, Tobias Grosser. *OOPSLA 2025*.
7. Interactive Bit Vector Reasoning using Verified Bitblasting. Henrik Böving, *Siddharth Bhat*, Alex Keizer, Luisa Cicolini, Abdalrhman Mohamed, Léo Stefanesco, Josh Clune, Clark Barrett, Tobias Grosser. *OOPSLA 2025*.

Full list (14 papers, incl. POPL, ICML, ICSE best paper, NeurIPS) on Google Scholar.

Software

Polly (LLVM): #3 contributor to LLVM's polyhedral loop optimizer: 121 commits, 6000 LoC, Fortran support and unified-memory CUDA codegen.
Lean 4: primary author of the LLVM backend (instruction selection, codegen); top-20 contributor overall.
Asterius / GHC: #2 contributor to this Haskell→WebAssembly compiler; reimplemented the GHC runtime on WASM, later merged into GHC proper.
lean-mlir: formal semantics for MLIR in Lean 4, with verified peephole rewriting.
PLUTO & PolyMage: loop-nest optimizers: found and fixed diamond-tiling bugs in PLUTO, added stencil and time-iterated-stencil support to PolyMage.
Simplexhc: a compiler for a lazy functional language applying polyhedral ideas to an LLVM backend.

Experience

2026–present XTX Markets, London — *ML Performance Engineer*. Performance engineering for machine-learning systems.
Sep–Nov '24 Amazon Web Services, Automated Reasoning Group, Austin — *Research Intern*. Deciding memory non-interference in `lnsym`, a Lean-based ARM symbolic simulator; large-scale symbolic execution and its performance.
Jul–Sep '23 Microsoft Research, Redmond — *Research Intern*. Retrieval-augmented proof synthesis for the F* proof assistant (ICSE 2025 best paper).
May–Jul '19 Tweag.io, Paris — *Research Intern*. Re-implemented portions of the GHC runtime for Asterius (Haskell, C, WASM).
Summer '18 Google Summer of Code, Polly Labs — *Mentor*. Enabling Polly's loop optimizations in Chapel.
Mar–Dec '17 ETH Zurich, Scalable Parallel Computing — *Research Intern*. CUDA code generation in Polly for Fortran loop nests.
May–Jul '16 IISc Bangalore — *Research Intern*. PolyMage DSL compiler; tiling patterns and stencil optimization; contributions to ISL and PLUTO.

Talks at LLVM Developer Meetings

How to trust your peephole rewrites: automatically verifying them for arbitrary width (EuroLLVM '25) · lean-mlir: verifying peephole optimizations in MLIR (LLVM Dev '24) · (Correctly) extending dominance to MLIR regions (LLVM Dev '23) · MLIR side-effect modelling (LLVM Dev '23) · MLIR for functional programming (EuroLLVM '22).

Education

PhD, Computer Science, University of Cambridge; transferred from University of Edinburgh — 2022–2026 (writing up)
MS, Computer Science, IIIT Hyderabad, India — 2020–2021
BTech, Computer Science, IIIT Hyderabad, India — 2015–2020